SOFTWARE ENGINEERING · CHAPTER 02

Lab-02: Spring Framework

Build one small application, **shop**, in five steps: bootstrap it with Maven and Spring Boot, wire it with dependency injection, persist with JDBC, expose a validated REST API, then add JSP pages, profiles and Actuator. The challenge turns it into a layered order-management system.

Spring Boot 4.1.1 · Spring Framework 7.0 · JDK 25 · Maven 3.9 · Gradle 9.7 · ≈ 5–6 hours · 5 tasks + 1 challenge

How to work through this lab. The tasks build on each other inside *one* project. Commit after every task (`git tag task-N`) so the grader can see each step. Tick the acceptance checklist as you go (it is saved in your browser). Open hints only after 10 minutes of being stuck.

0 Setup — 15 min

1. Verify the toolchain:

```
java -version      # 25
mvn -v             # Apache Maven 3.9.x (or use ./mvnw from the generated
project)
git --version
curl --version     # or use HTTPie / Postman / IntelliJ HTTP client
```

2. Generate the project at start.spring.io with these settings, or run the equivalent command:

```
curl https://start.spring.io/starter.zip \
  -d type=maven-project -d language=java -d javaVersion=25 \
  -d bootVersion=4.1.1 -d groupId=edu.itc.se -d artifactId=shop -d name=shop
\
  -d packageName=edu.itc.shop \
  -d dependencies=webmvc,validation,jdbc,h2,actuator,aop,devtools \
  -o shop.zip && unzip shop.zip -d shop && cd shop
```

   Dependencies: *Spring Web MVC, Validation, JDBC API, H2 Database, Actuator, Spring AOP, DevTools*. Spring Boot Test comes automatically.

3. Initialise git and make the first commit. Confirm `.gitignore` excludes `target/`.
4. Build once to download everything: `./mvnw -q verify` must print nothing but `BUILD SUCCESS` is implied by exit code 0.

5. Target package layout (create the packages as you go):

```
edu.itc.shop
├── ShopApplication.java
├── config/      ShopProperties, AppConfig
├── web/         HelloController, ProductController, ProductPageController,
ApiExceptionHandler
│   └── dto/     ProductDto, CreateProduct
├── service/     ProductService, Notifier + impls, TimingAspect
├── repository/  ProductRepository (JdbcClient), RawJdbcProductRepository
└── domain/      Product
```

1 Bootstrapping with Maven and Spring Boot

Easy · 30 min

GOAL

Understand what the generated project contains, run it three different ways, and see auto-configuration and the embedded server at work.

STEPS

1. Open `pom.xml`. In `README.md` answer: what does the `<parent>` give you? Why do the starters have no `<version>`? What does `spring-boot-maven-plugin` do at `package` time? Run `./mvnw dependency:tree | head -40` and paste the first level.
2. Create `web/HelloController` with `GET /hello?name=X` returning JSON `{"message":"Hello, X!","at":"2026-…"}` (a record `Greeting(String message, Instant at)`). Default name: `World`.
3. Run with `./mvnw spring-boot:run`. Call the endpoint with `curl -i`. Change the message text while it runs and watch DevTools restart the app.
4. Package: `./mvnw -q package`, then `java -jar target/shop-0.0.1-SNAPSHOT.jar --server.port=9090`. Confirm `/hello` answers on 9090. Note the jar size and explain it in `README.md`.
5. Call `GET /actuator/health`. Then start with `--debug` and find in the *condition evaluation report* the line that explains why `DataSourceAutoConfiguration` matched.
6. **Gradle bonus**: generate the same project with `type=gradle-project-kotlin` into a sibling folder and run `./gradlew bootRun`. Commit only the wrapper, not `.gradle/` or `build/`.

STARTER CODE

```
package edu.itc.shop.web;
```

```
import java.time.Instant;
import org.springframework.web.bind.annotation.*;

@RestController
public class HelloController {

    public record Greeting(String message, Instant at) {}

    @GetMapping("/hello")
    public Greeting hello(@RequestParam(defaultValue = "World") String name) {
        // TODO: return a Greeting
        return null;
    }
}
```

EXPECTED OUTPUT

```
$ curl -i "http://localhost:8080/hello?name=Ann"
HTTP/1.1 200
Content-Type: application/json

{"message":"Hello, Ann!","at":"2026-09-25T09:12:44.120Z"}

$ curl http://localhost:8080/actuator/health
{"status":"UP"}
```

Show hints

- Jackson serialises records automatically; `Instant` becomes an ISO-8601 string.
- DevTools only restarts when the IDE compiles or you run `./mvnw compile` in another terminal.
- The fat jar is ~25 MB because it embeds Tomcat, Spring and all starters; `jar tf target/*.jar | head` shows the `BOOT-INF/lib` layout.
- Search the debug report for `DataSourceAutoConfiguration matched`; the reason mentions `@ConditionalOnClass` for `DataSource` and the H2 driver.

ACCEPTANCE CHECKLIST

- [x] ~~`README.md` explains parent POM, version management and the Boot Maven plugin, with the dependency tree excerpt.~~
- [x] ~~`/hello` works from `spring-boot:run` and from the packaged jar on a different port.~~
- [x] ~~Actuator health responds and the condition evaluation line is quoted in the README.~~

- [x] ~~Tag `task-1` exists; `target/` is not committed.~~

2 Inversion of control, dependency injection and bean lifecycle

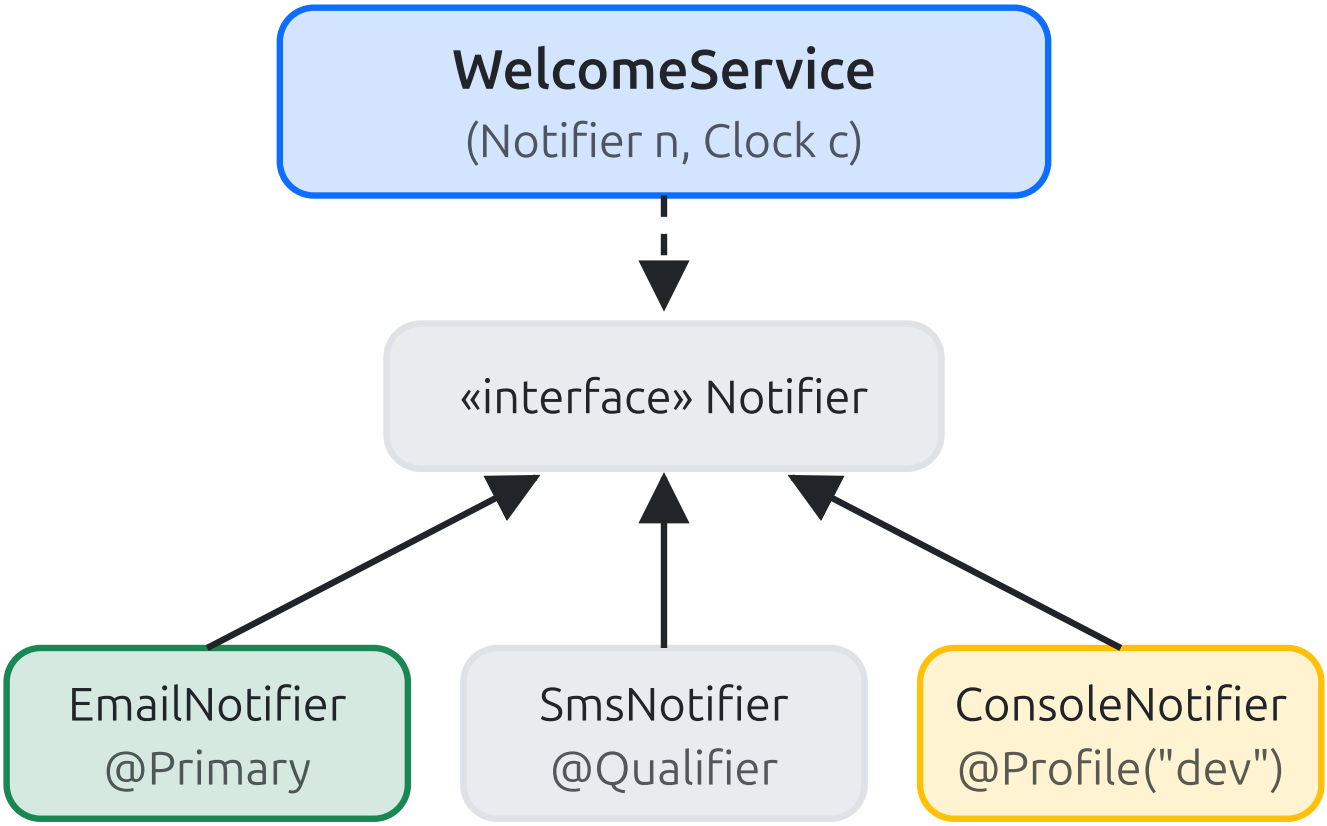
Easy **45 min**

GOAL

Let the container wire your objects: interfaces with several implementations, qualifiers, profiles, scopes and lifecycle callbacks, and a unit test that needs no Spring at all.

STEPS

1. Create `service/Notifier { void send(String to, String text); }` with two `@Service` implementations: `EmailNotifier` (logs `EMAIL to=… text=…`) and `SmsNotifier`. Mark `EmailNotifier` as `@Primary`.
2. Create `service/WelcomeService` with **constructor injection** of a `Notifier` and a `Clock`. Provide the `Clock` from an `@Bean` method in `config/AppConfig`.
3. Add a second constructor parameter `@Qualifier("smsNotifier") Notifier urgent` and a method `welcome(String user, boolean urgent)` choosing between them.
4. Add `@PostConstruct` / `@PreDestroy` logging to both notifiers. Add a `@Scope("prototype")` bean `ReportBuilder` and inject an `ObjectProvider<ReportBuilder>` somewhere; prove that `getObject()` returns a new instance every time (compare `System.identityHashCode`).
5. Create a third implementation `ConsoleNotifier` annotated `@Profile("dev")` and `@Primary`. Run with `--spring.profiles.active=dev` and without; observe which notifier is used. Then explain in `README.md` the rule that decides between two `@Primary` beans (what happens if both are active?).
6. Add a `CommandLineRunner` that prints the number of beans in the context and every bean name containing `Notifier`.
7. Write `WelcomeServiceTest` (plain JUnit 6, no Spring): pass a fake `Notifier` that records calls and a `Clock.fixed(...)`; assert the message text.



container resolves by type → @Primary → @Qualifier

STARTER CODE

```
package edu.itc.shop.service;

import java.time.Clock;
import java.time.LocalTime;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;

@Service
public class WelcomeService {
    private final Notifier defaultNotifier;
    private final Notifier urgentNotifier;
    private final Clock clock;

    public WelcomeService(Notifier defaultNotifier,
                          @Qualifier("smsNotifier") Notifier urgentNotifier,
                          Clock clock) {
        this.defaultNotifier = defaultNotifier;
        this.urgentNotifier = urgentNotifier;
        this.clock = clock;
    }
```

```
    public String welcome(String user, boolean urgent) {
        String text = "Welcome " + user + "! It is " +
LocalTime.now(clock).withNano(0);
        // TODO: pick the notifier, send, return the text
        return text;
    }
}
```

EXPECTED OUTPUT (START-UP, DEFAULT PROFILE)

```
INFO  e.i.shop.service.EmailNotifier  : EmailNotifier ready
INFO  e.i.shop.service.SmsNotifier    : SmsNotifier ready
INFO  e.i.shop.ShopApplication        : 312 beans; notifiers = [emailNotifier,
smsNotifier]
INFO  e.i.shop.service.EmailNotifier  : EMAIL to=ann text=Welcome ann! It is
09:15:02
INFO  e.i.shop.service.SmsNotifier    : SMS to=bob text=Welcome bob! It is
09:15:02
INFO  e.i.shop.ShopApplication        : prototype instances differ: true
...
INFO  e.i.shop.service.EmailNotifier  : EmailNotifier closing      ← on Ctrl+C
```

Show hints

- Default bean names are the class name with a lower-case first letter: `SmsNotifier` → `smsNotifier`.
- Inject `ApplicationContext ctx` into the runner and use `ctx.getBeanDefinitionCount()` and `ctx.getBeanNamesForType(Notifier.class)`.
- Two active `@Primary` candidates of the same type cause `NoUniqueBeanDefinitionException` at start-up. Fix it by giving `ConsoleNotifier` the profile *and* excluding `EmailNotifier` with `@Profile("!dev")`.
- The fake notifier in the unit test can be a lambda: `Notifier fake = (to, text) -> calls.add(to + ":" + text);`

ACCEPTANCE CHECKLIST

- [x] ~~All beans use constructor injection with `final` fields; no field injection anywhere.~~
- [x] ~~`@Primary`, `@Qualifier` and `@Profile` each demonstrably change which implementation is used.~~
- [x] ~~Lifecycle callbacks log on start-up and shutdown; prototype scope is proven with identity hash codes.~~
- [x] ~~`WelcomeServiceTest` passes without starting a Spring context.~~

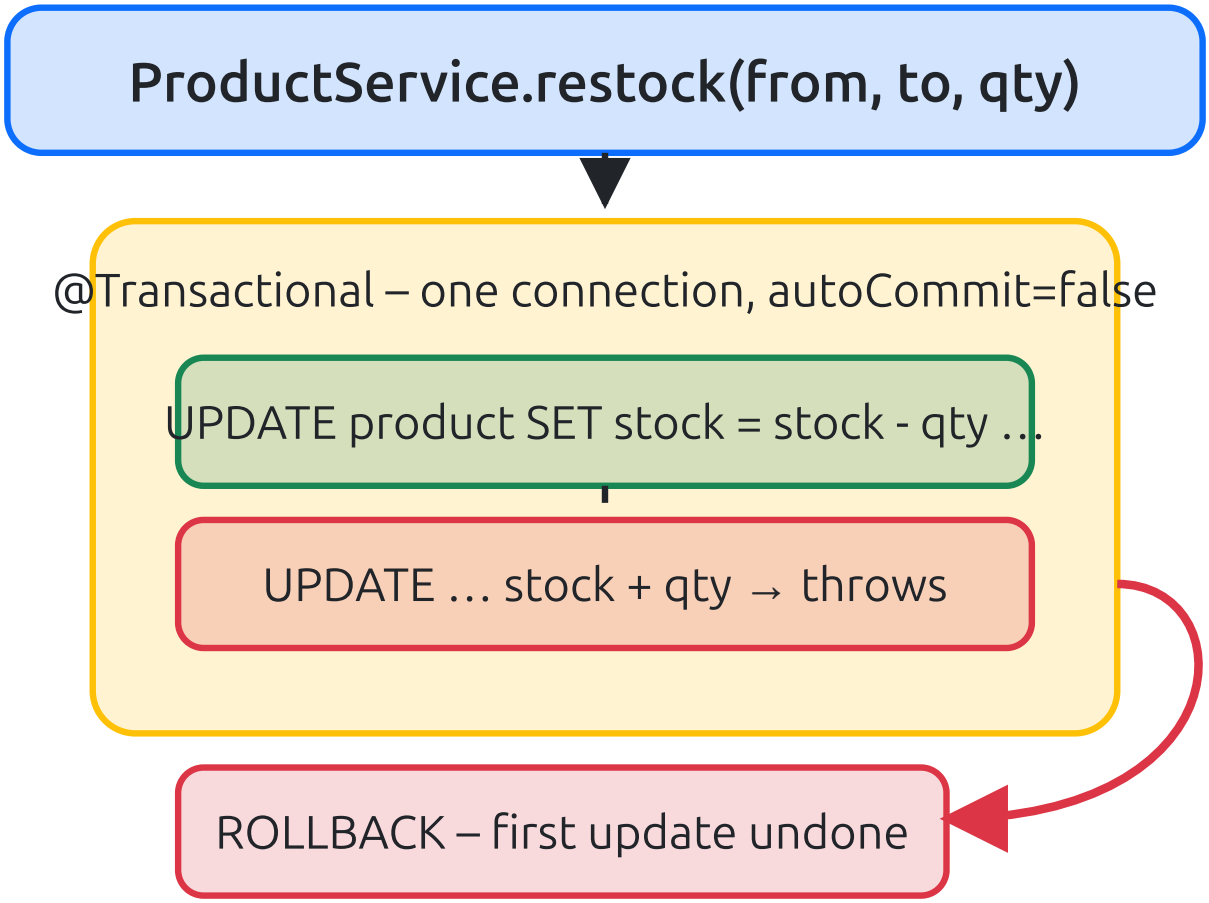
3 JDBC, `JdbcClient` and transactions **Medium** **60 min**

GOAL

Persist products in an H2 database: first with raw JDBC to feel the pain, then with `JdbcClient`, and wrap a multi-statement operation in a declarative transaction that rolls back on failure.

STEPS

1. Create `domain/Product(Long id, String name, BigDecimal price, int stock)` as a record.
2. Add `src/main/resources/schema.sql` (table `product` with identity PK, `name VARCHAR(80) NOT NULL UNIQUE`, `price DECIMAL(10,2)`, `stock INT CHECK (stock >= 0)`) and `data.sql` with five products. Configure H2 in `application.yml` and enable the H2 console.
3. Write `repository/RawJdbcProductRepository.findAll()` using **only** `DataSource`, `PreparedStatement` and `ResultSet` with try-with-resources. Count the lines.
4. Write `repository/ProductRepository` (`@Repository`) with `JdbcClient`: `findAll()`, `findById(long)` → `Optional`, `findByNameContaining(String)`, `insert(Product)` returning the generated id, `updateStock(long id, int delta)` returning affected rows, `delete(long)`.
5. Write `service/ProductService` with `@Transactional restock(long fromId, long toId, int qty)`: subtract from one product, add to the other. Throw `IllegalStateException` when the first update affects 0 rows (or the CHECK constraint fails). Make sure the whole operation rolls back.
6. Prove it with `ProductServiceTest` annotated `@SpringBootTest`: call `restock` with too large a quantity, assert the exception, then assert both stocks are unchanged. Add a second test class annotated `@JdbcTest` + `@Import(ProductRepository.class)` for the repository.
7. Insert a product with a duplicate name and observe which exception reaches you. Note its class in `README.md` and explain where the translation happened.



STARTER CODE

```
package edu.itc.shop.repository;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.support.GeneratedKeyHolder;
import org.springframework.stereotype.Repository;
import edu.itc.shop.domain.Product;

@Repository
public class ProductRepository {
    private final JdbcClient jdbc;
    public ProductRepository(JdbcClient jdbc) { this.jdbc = jdbc; }

    public List<Product> findAll() {
        return jdbc.sql("SELECT id, name, price, stock FROM product ORDER BY
id")
                   .query(Product.class).list();
    }
```

```
    public Optional<Product> findById(long id) {
        // TODO
        return Optional.empty();
    }

    public long insert(Product p) {
        var keys = new GeneratedKeyHolder();
        jdbc.sql("INSERT INTO product(name, price, stock) VALUES (:name, :price,
:stock)")
            .paramSource(p)
            .update(keys);
        return keys.getKey().longValue();
    }

    public int updateStock(long id, int delta) {
        // TODO: UPDATE product SET stock = stock + :delta WHERE id = :id AND
stock + :delta >= 0
        return 0;
    }
}


@SpringBootTest
class ProductServiceTest {
    @Autowired ProductService service;
    @Autowired ProductRepository repo;

    @Test
    void restockRollsBackWhenSourceHasTooLittleStock() {
        int before1 = repo.findById(1).orElseThrow().stock();
        int before2 = repo.findById(2).orElseThrow().stock();

        assertThrows(IllegalStateException.class, () -> service.restock(1, 2,
1_000));

        assertEquals(before1, repo.findById(1).orElseThrow().stock());
        assertEquals(before2, repo.findById(2).orElseThrow().stock());   //
unchanged thanks to rollback
    }
}
```

EXPECTED OUTPUT

```
$ ./mvnw -q test
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0

# with logging.level.org.springframework.jdbc=DEBUG and
```

```
org.springframework.transaction=DEBUG
DEBUG o.s.j.d.DataSourceTransactionManager : Creating new transaction with name
[...ProductService.restock]
DEBUG o.s.jdbc.core.JdbcTemplate           : Executing prepared SQL statement
[UPDATE product SET stock = ...]
DEBUG o.s.j.d.DataSourceTransactionManager : Initiating transaction rollback
DEBUG o.s.j.d.DataSourceTransactionManager : Rolling back JDBC transaction on
Connection [HikariProxyConnection@...]
```

Show hints

- `spring.sql.init.mode=always` is needed only for non-embedded DBs; H2 in-memory runs the scripts by default.
- Record mapping: column names `id, name, price, stock` must match component names (snake_case → camelCase is handled).
- To make the second update fail deterministically, target a non-existent `toId` in another test and check that the first update was rolled back too.
- The duplicate-name insert surfaces as `DuplicateKeyException` (a `DataAccessException`): translation happens in the proxy that `@Repository` adds.
- Do not call `restock` from another method inside `ProductService`; the self-invocation would skip the transaction.

ACCEPTANCE CHECKLIST

- [ ] Raw JDBC and JdbcClient versions both work; the line-count comparison is in `README.md`.
- [ ] All SQL uses bound parameters; no string concatenation of user input.
- [ ] `restock` is atomic: the rollback test passes and the transaction log lines are visible.
- [ ] `@JdbcTest` repository tests and the duplicate-key explanation are present.

4 REST API with Spring MVC, validation and error handling

Medium **60 min**

GOAL

Expose products over HTTP with correct status codes, validated DTOs and a single, consistent error format, then test the controller in isolation with MockMvc.

STEPS

1. Create DTO records in `web/dto`: `ProductDto(long id, String name, BigDecimal price, int stock)` with a static `from(Product)`, and `CreateProduct(@NotBlank @Size(max=80) String name, @NotNull @DecimalMin("0.01") BigDecimal price, @PositiveOrZero int stock)`.
2. Implement `web/ProductController` at `/api/products`:
   - `GET` list, optional `?q=` filter (name contains, case-insensitive)
   - `GET /{id}` → 200 or 404
   - `POST` → 201 with `Location: /api/products/{id}` header and the created body
   - `PATCH /{id}/stock` body `{"delta": -2}` → 200 with updated product, 409 if stock would go negative
   - `DELETE /{id}` → 204, 404 if missing
3. Create `ProductNotFoundException` and `InsufficientStockException` in `service`; controllers must not contain `try/catch`.
4. Create `web/ApiExceptionHandler` (`@RestControllerAdvice`) returning `ProblemDetail` for: validation errors (400, with a list of field errors), not found (404), insufficient stock (409), `DuplicateKeyException` (409), and a generic fallback (500 that does *not* leak the stack trace).
5. Write `ProductControllerTest` with `@WebMvcTest(ProductController.class)`, `MockMvc` and a `@MockitoBean ProductService`. Cover: 200 list, 404 by id, 400 with two field errors, 201 with Location header.
6. Record a session of `curl` calls (or an `.http` file) exercising every endpoint including the error cases; commit it as `api-session.md`.

STARTER CODE

```
package edu.itc.shop.web;

import java.net.URI;
import java.util.List;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import edu.itc.shop.service.ProductService;
import edu.itc.shop.web.dto.*;

@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService service;
    public ProductController(ProductService service) { this.service = service; }

    @GetMapping
```

```
    public List<ProductDto> list(@RequestParam(required = false) String q) {
        return service.search(q);
    }

    @PostMapping
    public ResponseEntity<ProductDto> create(@RequestBody @Valid CreateProduct
cmd) {
        ProductDto created = service.create(cmd);
        return ResponseEntity.created(URI.create("/api/products/" +
created.id())).body(created);
    }

    // TODO: GET /{id}, PATCH /{id}/stock, DELETE /{id}
}


@WebMvcTest(ProductController.class)
class ProductControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean ProductService service;

    @Test
    void createWithInvalidBodyReturns400WithFieldErrors() throws Exception {
        mvc.perform(post("/api/products")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                        {"name": "", "price": 0, "stock": -1}
                        """))
           .andExpect(status().isBadRequest())
           .andExpect(content().contentType(MediaType.APPLICATION_PROBLEM_JSON))
           .andExpect(jsonPath("$.errors.length()").value(3));
    }
}
```

EXPECTED OUTPUT

```
$ curl -i -X POST localhost:8080/api/products -H 'Content-Type:
application/json' \
       -d '{"name":"USB-C cable","price":7.90,"stock":25}'
HTTP/1.1 201
Location: /api/products/6
Content-Type: application/json

{"id":6,"name":"USB-C cable","price":7.90,"stock":25}

$ curl -i -X PATCH localhost:8080/api/products/6/stock -H 'Content-Type:
application/json' -d '{"delta":-30}'
```

```
HTTP/1.1 409
Content-Type: application/problem+json

{"type":"about:blank","title":"Conflict","status":409,
 "detail":"Product 6 has 25 units, cannot remove
30","instance":"/api/products/6/stock"}

$ curl -i localhost:8080/api/products/999
HTTP/1.1 404
{"type":"about:blank","title":"Not Found","status":404,"detail":"Product 999 not
found","instance":"/api/products/999"}
```

Show hints

- In Spring Boot 4 the MockMvc slice-test support lives in the `spring-boot-webmvc-test` module; `spring-boot-starter-test` already includes it when the web starter is present. `@MockitoBean` replaced the old `@MockBean`.
- Field errors: `ex.getBindingResult().getFieldErrors()`; put them in `pd.setProperty("errors", list)`.
- For the generic 500 handler log the exception with `log.error(...)` and return only `"Unexpected error, reference " + UUID`.
- For `PATCH` create a tiny record `StockChange(@NotNull Integer delta)` and validate it too.
- Use `jsonPath("$.errors[*].field", containsInAnyOrder("name","price","stock"))` from Hamcrest for a stronger assertion.

ACCEPTANCE CHECKLIST

- [ ] All five endpoints return the specified status codes and headers.
- [ ] Every error is an `application/problem+json` body produced by the single advice class; no stack traces leak.
- [ ] Controllers contain no business logic and no `try/catch`.
- [ ] Four MockMvc tests pass with a mocked service; `api-session.md` is committed.

5 JSP views, profiles, Actuator, logging and an aspect Hard

75 min

GOAL

Add a server-rendered UI with JSP, make the app configurable per environment, and make it operable: health checks, structured logging and a timing aspect.

STEPS

1. **JSP.** Change `<packaging>` to `war`; add `org.apache.tomcat.embed:tomcat-embed-jasper` (scope `provided` is fine for embedded runs) and `org.glassfish.web:jakarta.servlet.jsp.jstl`. Configure `spring.mvc.view.prefix=/WEB-INF/jsp/` and `suffix=.jsp`.
2. Create `web/ProductPageController` (`@Controller`) with `GET /products` (list + search box) and `GET /products/new` + `POST /products` (form with `@Valid @ModelAttribute`, `BindingResult`, POST-redirect-GET, flash message). Write `list.jsp` and `form.jsp` using JSTL (`c:forEach`, `c:out`, `fmt:formatNumber`) and Spring's form error rendering (`<form:errors>` from the `spring-form` taglib or manual `${errors}`).
3. **Profiles.** Create `application-dev.yml` (H2 console on, DEBUG logging for `edu.itc.shop` and SQL) and `application-prod.yml` (no console, INFO, datasource URL/credentials from `${DB_URL}` etc.). Add `config/ShopProperties` (`@ConfigurationProperties("shop")` record, validated) with `currency`, `taxRate` and `lowStockThreshold`; show the currency and gross price (price × (1 + taxRate)) on the JSP page.
4. **Actuator.** Expose `health,info,metrics,loggers`. Implement `LowStockHealthIndicator`: DOWN with detail `lowStockProducts` when any product is below `shop.low-stock-threshold`. Fill `info.app.*` from Maven properties.
5. **Logging.** Add a servlet `Filter` that puts a random `requestId` into the MDC and removes it afterwards; configure the console pattern to show it. Log every product creation at INFO and every failed validation at WARN.
6. **Aspect.** Add `service/TimingAspect` with an `@Around` advice on all public service methods logging `<method> took <µs>` at DEBUG. Then change the log level of the aspect's logger *at runtime* via `POST /actuator/loggers/edu.itc.shop.service.TimingAspect` and show the effect.
7. Run the app twice: `--spring.profiles.active=dev` and `prod` (with env vars for the DB). Capture the start-up banner lines showing the active profile and the H2 console availability difference in `README.md`.

STARTER CODE

```
<%-- src/main/webapp/WEB-INF/jsp/products/list.jsp --%>
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib prefix="c"   uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
<!DOCTYPE html>
<html><head><title>Products</title><link rel="stylesheet" href="/css/site.css">
</head>
<body>
<h1>Products (${currency})</h1>
<c:if test="${not empty message}"><p class="flash"><c:out value="${message}"/>
```

```
</p></c:if>
<form method="get"><input name="q" value="<c:out value='${q}'/>"
placeholder="search"><button>Go</button></form>
<table>
  <tr><th>Name</th><th>Net</th><th>Gross</th><th>Stock</th></tr>
  <c:forEach var="p" items="${products}">
    <tr class="${p.stock < threshold ? 'low' : ''}">
      <td><c:out value="${p.name}"/></td>
      <td><fmt:formatNumber value="${p.price}" minFractionDigits="2"/></td>
      <td><fmt:formatNumber value="${p.gross}" minFractionDigits="2"/></td>
      <td>${p.stock}</td>
    </tr>
  </c:forEach>
</table>
<a href="/products/new">Add product</a>
</body></html>
```

```
@Component
public class RequestIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse
res, FilterChain chain)
            throws ServletException, IOException {
        MDC.put("requestId", UUID.randomUUID().toString().substring(0, 8));
        try { chain.doFilter(req, res); }
        finally { MDC.remove("requestId"); }      // the thread is reused by the
pool!
    }
}
```

EXPECTED OUTPUT

```
$ ./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
... The following 1 profile is active: "dev"
... H2 console available at '/h2-console'. Database available at
'jdbc:h2:mem:shop'
10:41:07.221 INFO  [a3f9c2d1] e.i.shop.service.ProductService : created product
6 'USB-C cable'
10:41:07.223 DEBUG [a3f9c2d1] e.i.shop.service.TimingAspect   :
ProductService.create(..) took 1840 µs

$ curl localhost:8080/actuator/health
{"status":"DOWN","components":{"db":{"status":"UP"},"lowStock":
{"status":"DOWN","details":{"lowStockProducts":["Keyboard"]}}}}

$ curl -X POST localhost:8080/actuator/loggers/edu.itc.shop.service.TimingAspect
```

```
  \
        -H 'Content-Type: application/json' -d '{"configuredLevel":"OFF"}'
→ timing lines disappear without restart
```

Show hints

- JSPs are found from `src/main/webapp` when running with `spring-boot:run` or from the IDE; for `java -jar` you need the `war` packaging (Boot builds an executable war).
- Compute `gross` in the service or in a small view model record `ProductRow(name, price, gross, stock)`; keep arithmetic out of the JSP.
- `@ConfigurationProperties` records need `@ConfigurationPropertiesScan` on the application class (or `@EnableConfigurationProperties`).
- `HealthIndicator` bean names drop the suffix: `LowStockHealthIndicator` shows up as `lowStock`.
- Logback pattern: `%d{HH:mm:ss.SSS} %-5level [%X{requestId}] %logger{36} : %msg%n`.
- The aspect needs `spring-boot-starter-aop`; verify it applies by checking that the service bean's class name contains `$$SpringCGLIB$$`.

ACCEPTANCE CHECKLIST

- [ ] JSP list and form pages work, escape output, show validation errors and use POST-redirect-GET.
- [ ] `dev` and `prod` profiles behave differently; no secret is committed; `ShopProperties` is validated at start-up.
- [ ] Custom health indicator, info endpoint and runtime log-level change are demonstrated.
- [ ] Request id appears in every log line of a request; the timing aspect logs service calls.

★ Challenge: layered order management Hard

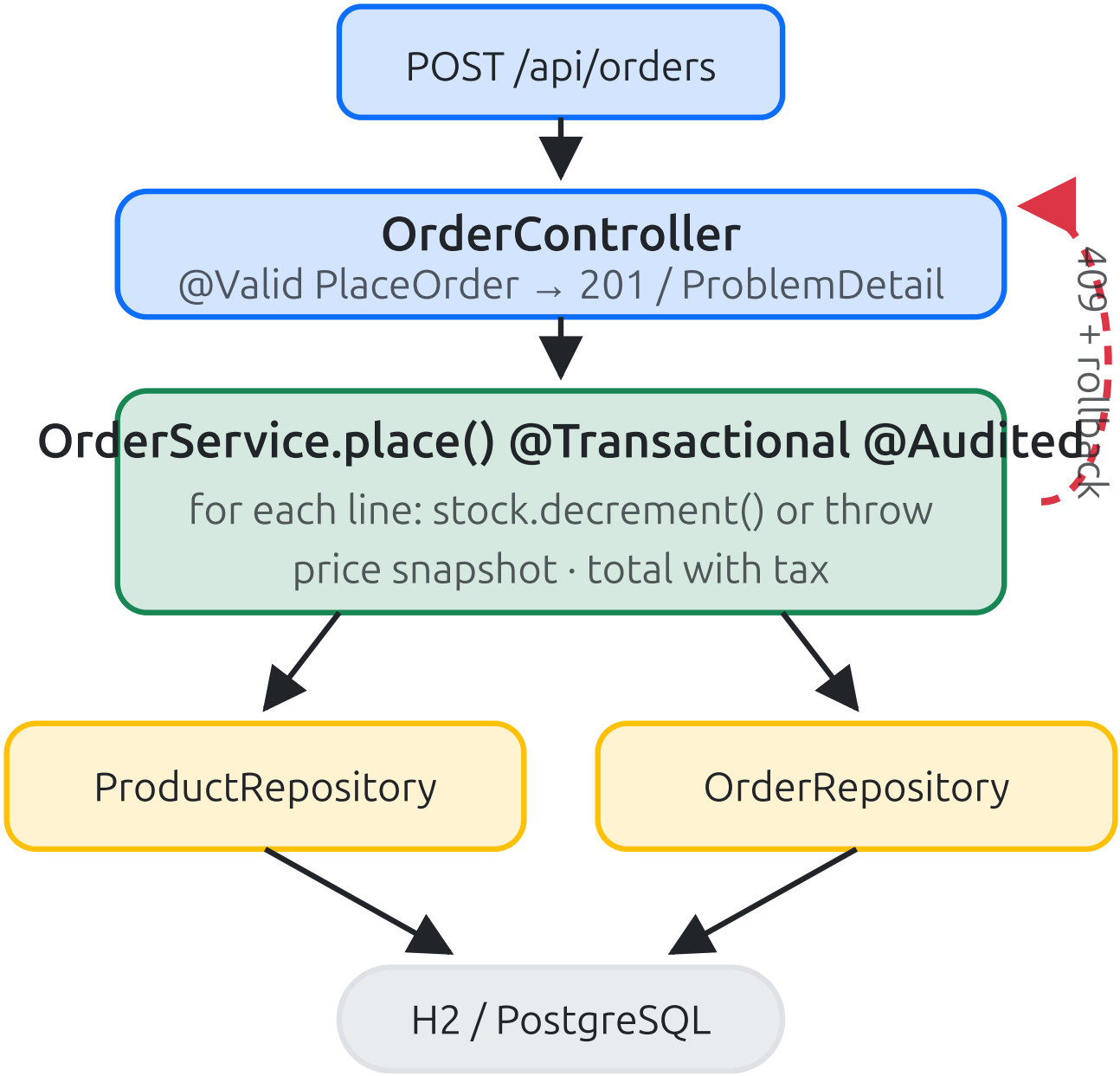
120 min · optional, extra credit

SCENARIO

The shop needs orders. A customer places an order with several lines; stock must be reserved atomically for all lines or none; orders can be listed with pagination and cancelled (which returns stock). The code must follow the layered architecture from the chapter and be covered by tests at every layer.

REQUIREMENTS

1. **Domain & schema.** Tables `customer`, `orders` (`id, customer_id, status, placed_at, total`) and `order_line` (`order_id, product_id, qty, unit_price`). Status enum `NEW, PAID, CANCELLED`. Repositories with `JdbcClient`; package **by feature** (`product/`, `order/`, `customer/`), each with its own web, service and repository classes.
2. **Placing an order.** `POST /api/orders` with `{customerId, lines:[{productId, qty}]}`: validate (at least one line, qty ≥ 1, no duplicate product), decrement stock for every line inside one `@Transactional` method, snapshot the unit price, compute the total with `ShopProperties.taxRate`, return 201. Any line without enough stock → 409 and nothing is changed.
3. **Reading.** `GET /api/orders?customerId=&status=&page=&size=` returning `{content:[…], page, size, totalElements}` using SQL `LIMIT/OFFSET`; `GET /api/orders/{id}` with lines.
4. **Cancelling.** `POST /api/orders/{id}/cancel`: only `NEW` orders; restores stock; otherwise 409. Sends a notification through the `Notifier` from Task 2.
5. **Cross-cutting.** An `@Audited` annotation + aspect that logs who/what/when for every audited service method; a `request`-scoped bean holding the current `requestId` injected into the aspect.
6. **Configuration.** A `test` profile using H2 with a separate `data-test.sql`; `prod` profile expecting PostgreSQL (provide a `docker-compose.yml`; PostgreSQL driver as `runtimeOnly`).
7. **Tests.** Unit tests for the pricing logic (no Spring); `@JdbcTest` for repositories; `@WebMvcTest` for controllers; one `@SpringBootTest(webEnvironment = RANDOM_PORT)` end-to-end test with `RestTestClient`/`TestRestTemplate` that places an order, checks the stock, cancels it and checks again.
8. **Report.** `README.md` with a component/package diagram, the request flow for "place order" through all layers, and a paragraph on where the transaction boundary is and why.



SKELETON

```
package edu.itc.shop.order;

public record PlaceOrder(@NotNull Long customerId,
                         @NotEmpty @Valid List<Line> lines) {
    public record Line(@NotNull Long productId, @Min(1) int qty) {}
}

@Service
public class OrderService {
    private final OrderRepository orders;
    private final ProductRepository products;
    private final ShopProperties props;
    private final Notifier notifier;
```

```
    /* constructor … */

    @Transactional
    @Audited
    public OrderDto place(PlaceOrder cmd) {
        // 1. validate no duplicate productId
        // 2. for each line: products.updateStock(id, -qty) == 0 → throw
InsufficientStockException
        // 3. load unit prices, compute total (net × (1 + taxRate)), round
HALF_UP to 2 decimals
        // 4. orders.insert(order, lines) → id
        // 5. return OrderDto
        return null;
    }

    @Transactional
    @Audited
    public OrderDto cancel(long id) {
        // only NEW → CANCELLED, restore stock, notifier.send(...)
        return null;
    }

    @Transactional(readOnly = true)
    public Page<OrderDto> list(Long customerId, OrderStatus status, int page,
int size) { return null; }
}
```

EXPECTED OUTPUT

```
$ curl -i -X POST localhost:8080/api/orders -H 'Content-Type: application/json'
\
   -d '{"customerId":1,"lines":[{"productId":1,"qty":2},
{"productId":3,"qty":1}]}'
HTTP/1.1 201
Location: /api/orders/17
{"id":17,"status":"NEW","total":131.88,"lines":
[{"productId":1,"qty":2,"unitPrice":49.90},
{"productId":3,"qty":1,"unitPrice":10.00}]}

$ curl -i -X POST localhost:8080/api/orders -H 'Content-Type: application/json'
\
   -d '{"customerId":1,"lines":[{"productId":1,"qty":2},
{"productId":2,"qty":999}]}'
HTTP/1.1 409
{"title":"Conflict","status":409,"detail":"Insufficient stock for product 2"}
# product 1 stock is unchanged – the first decrement was rolled back
```

```
10:52:13.004 INFO [b71e0c9a] e.i.shop.audit.AuditAspect : AUDIT
OrderService.place customer=1 result=order 17 (3 ms)
$ ./mvnw -q verify   → 0 failures across unit, slice and end-to-end tests
```

Show hints

- Make `updateStock` conditional in SQL (`AND stock + :delta >= 0`) and check the affected-row count; this is race-free without explicit locking (compare with Chapter 01).
- Insert lines with `JdbcClient` in a loop, or use `NamedParameterJdbcTemplate.batchUpdate` for a batch.
- Pagination: run a `COUNT(*)` with the same `WHERE` clause; build the clause dynamically only from *whitelisted* filters, never from raw strings.
- Request-scoped beans injected into singletons need a proxy: `@RequestScope` already sets `proxyMode = TARGET_CLASS`.
- For the end-to-end test use the `test` profile with `@ActiveProfiles("test")` and reset data with `@Sql` scripts per test.
- Draw the diagram in Mermaid inside `README.md`; GitHub and GitLab render it.

ACCEPTANCE CHECKLIST

- [ ] Placing and cancelling orders are atomic; the 409 case leaves stock untouched (proven by a test).
- [ ] Pagination, filtering and both profiles work; PostgreSQL runs via the provided docker-compose.
- [ ] Audit aspect and request-scoped bean log every audited call with the request id.
- [ ] Tests exist at all four levels and `README.md` contains the diagrams and the transaction-boundary explanation.

Grading rubric

Component	Points	Full marks when…
Task 1 · Bootstrapping	10	Runs three ways, README explains the build, condition report quoted.
Task 2 · IoC & DI	15	Constructor injection everywhere, qualifiers/profiles/scopes demonstrated, Spring-free unit test.

Component	Points	Full marks when…
Task 3 · JDBC & transactions	20	Both repository styles, bound parameters, rollback proven by test, exception translation explained.
Task 4 · REST API	20	Correct status codes, validated DTOs, ProblemDetail advice, MockMvc tests, API session.
Task 5 · Views, profiles, operations	25	JSP pages, profiles, typed properties, health indicator, MDC logging, aspect and runtime log level.
Code quality	10	Layered packages, no logic in controllers, no secrets in git, wrapper committed, tags per task.
★ Challenge (extra credit)	+20	All acceptance items ticked; tests at every layer pass; README diagrams present.
Total	**100 (+20)**	

Automatic deductions: −5 per field-injected dependency; −5 for SQL built by string concatenation with user input; −10 if a controller catches exceptions or contains business rules; −10 if `@Transactional` is on a controller or repository method instead of the service; −10 for committed passwords or `target/`.

Submission

1. Repository layout:

```
shop/
├── pom.xml  ·  mvnw  ·  .mvn/
├── README.md                ← answers, dependency tree, profile output,
diagrams
├── api-session.md           ← curl / .http transcript from Task 4
├── docker-compose.yml       ← challenge only (PostgreSQL)
└── src/
    ├── main/java/edu/itc/shop/…
    ├── main/resources/  application.yml · application-dev.yml ·
application-prod.yml · schema.sql · data.sql · static/css/site.css
    ├── main/webapp/WEB-INF/jsp/products/  list.jsp · form.jsp
    └── test/java/edu/itc/shop/…
```

2. `./mvnw -q verify` must succeed from a clean checkout on JDK 25 without any environment variables (tests use the `test` or default profile with H2).

3. Tags `task-1` … `task-5` (and `challenge`) must exist. Push branch `lab-02` and open a merge request titled `Lab-02 – <your name>`.
4. Include screenshots of the JSP product list page and of `/actuator/health` in DOWN state.

Back to Chapter 02 slides | Retake the chapter quiz | ← Lab-01